compared to code written without tests, your code has more surface area and less work happening behind the scenes where it's hard to observe.

This theory explains why testing works better when the tests come first. Even waiting a little bit to write tests is significantly more painful. When the tests are written first, in very close intertwined proximity to the code, they encourage a good structure with low coupling (meaning different parts of the code have minimal dependencies on each other) and high cohesion (meaning code that is in the same unit is all related).

When the tests come later, they have to conform to the existing code, and it's amazing how quickly code written without tests will move toward low-cohesion and high-coupling forms that are much harder to cover with tests. If your only experience is with writing automated tests long after the initial code was written, the experience was likely quite painful. Don't let that turn you away from a TDD approach; the tests and code you'll write with TDD are much different.

When you're writing truly test-driven code, the tests are the final source of truth in your application. This means that when there is a discrepancy between the code and the tests, your first assumption is that the test is correct and the code is wrong. If you're writing tests after the code, then your assumption must be that the code is the source of truth. As you write your code using test-driven development, keep in mind the idea that the tests are the source of truth and are guiding the code's structure.

Prescription 2	In a test-driven process, if it's difficult to write tests for a feature, strongly consider the possibility that the design of the underlying code needs to be changed.

What Is TDD Good For?

The primary purpose of test-driven development is to go beyond mere verification and use the tests to improve the code's structure. That is, TDD is a software-development technique masquerading as a code-verification tool.

Automated tests are a wonderful way of showing that the program does what the developer thinks it does, but they are a lousy way of showing that what the developer thinks is what the program actually should do. "But the tests pass!" is not likely to be comforting to a customer when the developer's assumptions are just flat-out wrong. I speak from painful experience.

The kinds of tests written in a TDD process are not a substitute for acceptance testing, where users or customers verify that the code does what the user or